



Setting up Immutable Infrastructure Using Hashicorp Terraform and Jenkins

Infrastructure as Code has gained much popularity nowadays because of its easy implementation and the possibility of building clean infrastructure with a declarative programming model. This article covers the various approaches of building and maintaining your infrastructure with Terraform and Jenkins Server.

DevOps methodologies and practices have transformed the complexities of IT infrastructure management into code that manages the entire IT infrastructure with just a little maintenance. We have a lot of configuration management tools and orchestration tools to tailor IT Infrastructure as Code (IaC), but selecting the right tool depends on numerous factors. One needs to analyse the pros and cons of the tool, understand how it fits a particular use case, and check if its source code has been fully open sourced since there should be no vendor lock-in. There should also be clear official documentation, good community support, easy integration with the platform, and interoperability with different cloud solutions and third party software.

Common scenarios in IT infrastructure management

Provisioning and de-provisioning resources in a cloud environment is a common practice for testing and releasing a software product without any bugs. In conjunction with continuous integration (CI) and continuous deployment (CD) tools, we may need to use both orchestration tools and configuration management solutions. In any cloud environment, orchestration tools such as Terraform, CloudFormation, Heat and Azure Resource Manager are responsible for provisioning infrastructure. Configuration management tools such as Chef, Ansible, Puppet and SaltStack take care of installing the software packages on the server, configuring the services, and deploying the applications on top of it. But today, configuration management tools have some amount of support for provisioning resources in the cloud as well as the support of provisioning tools for installing and configuring software

on newly created resources. This balances the complexity of provisioning and managing infrastructure. On the other hand, it is difficult to achieve everything with a single tool. The recommended way is to use both provisioning and configuration tools to manage infrastructure at scale.

The need for immutable infrastructure

Even though we manage infrastructure with configuration management tools, there are chances of 'configuration drift' in the servers if there are frequent changes applied to the server. In order to avoid this, we should neither change the configuration of a running server by modifying it manually nor through configuration management tools. Maintaining immutable infrastructure is a best practice to avoid configuration drift.

Immutable infrastructure is now becoming a popular term across the DevOps community. It is the practice of provisioning a new server for every config change and de-provisioning old ones. Provisioning tools like Terraform and CloudFormation support the creation of immutable infrastructure to a great extent. For every software configuration change, these tools help to create new infrastructure and deploy the new configuration before deleting the old one. This helps to manage large infrastructure, as we do not need to worry about the configuration changes and their impact over a period of time.

In a production environment, DevOps practitioners often follow Blue-Green deployment to avoid unexpected issues that lead to downtime in production. Rollback is possible in such a deployment, and the application can go back into the previous state without any difficulties because no changes have been made to the existing environment.